

Capítulo 7: Estruturação de Relatórios

Gabriel Sanábio Mazzetti Affonso

2026-09-21

Índice

Prefácio	4
1 Introdução	5
1.1 A importância do Python na Estatística	5
1.2 O escopo deste material	5
2 Configurando o Ambiente de Trabalho	7
2.1 O Ambiente <i>Python</i> e o pacote <i>reticulate</i>	7
2.2 Gerenciamento de Bibliotecas com o <i>pip</i>	7
2.3 Instalação Direta em Blocos de Código	7
3 Fundamentos	9
3.1 Filosofia, Sintaxe e Tipagem da Linguagem	9
3.2 Estruturas de Dados Nativas	10
3.3 Estruturas de Controle de Fluxo	10
3.4 Funções e Ferramentas Funcionais	11
3.5 Aplicação Prática: Manipulação de Listas	11
4 Análise Exploratória e Manipulação de Dados em <i>Python</i>	13
4.1 A Biblioteca <i>Pandas</i> e as Estruturas de Dados	13
4.2 Importação e Tratamento de Dados (<i>Data Wrangling</i>)	14
4.3 Visualização dos Dados	14
4.4 Estudos de Caso Práticos	15
4.4.1 Análise de Dados Categóricos: <i>Titanic</i>	15
4.4.2 Análise de Dados Contínuos: <i>Iris</i>	17
5 Computação Numérica com NumPy	20
5.1 A Revolução Vetorial e o Objeto	20
5.2 Criação e Formatação de Matrizes	21
5.3 Álgebra, <i>Broadcasting</i> e Indexação Booleana	22
5.4 Funções Estatísticas e Simulação Aleatória	24
6 Web Scraping	27
6.1 Fundamentos da <i>Web</i> e Ética na Extração	27
6.1.1 Ética e Boas Práticas na Raspagem	28
6.2 O Ecossistema de Raspagem Estática	28
6.3 Navegação e Extração de Elementos (Conteúdo Estático)	29
6.4 Raspagem de Estruturas Tabulares (Integração com <i>Pandas</i>)	30
6.5 Raspagem Dinâmica e Automação com Selenium	32
6.5.1 Configuração do Webdriver	32
6.5.2 Interação com a Página	33

6.5.3	Espera Explícita e Implícita	33
6.6	Estudos de Caso Completos	34
6.6.1	Caso 1 (Estático): Extraindo uma Tabela Real com Requests e BeautifulSoup	34
6.6.2	Caso 2 (Dinâmico): Automatizando a Coleta com Selenium	35
7	Livros Digitais: Estruturação de relatórios dinâmicos via <i>Quarto</i>	37
7.1	A Evolução para o <i>Quarto</i>	37
7.2	A Estrutura de Metadados (O Cabeçalho <i>YAML</i>)	37
7.3	<i>Templates</i> e Padronização Visual	38
7.4	Elementos Científicos e Tabelas Complexas	38
7.5	Compilação de Livros Digitais (O arquivo <code>_quarto.yml</code>)	39
8	Referências	41

Prefácio

O presente material consiste na consolidação dos resultados de pesquisa obtidos durante o período de Iniciação Científica, no contexto do projeto **Métodos de Aprendizado Estatístico para Discriminação e Classificação: Aplicações à Indústria 4.0 – Fase 3**. Este documento visa não só o detalhamento dos métodos de coleta de dados via *Web Scraping*, mas também a viabilidade do uso do *Python* para tal, além de destacar as especificidades da linguagem e suas vantagens para o aprendizado de métodos estatísticos e computacionais.

Neste contexto, vale ressaltar que o material é de cunho introdutório, buscando viabilizar o aprendizado do *Python* enquanto linguagem de programação e do *Web Scraping* como técnica poderosa de coleta de dados. Dessa forma, seu foco é desenvolver, no estatístico, o interesse pelos diversos recursos computacionais que podem auxiliá-los enquanto profissionais e estudiosos da área..

A motivação para o desenvolvimento deste material surgiu da necessidade de diversificar as ferramentas de aplicação computacional existentes para analisar dados. A interseção entre a teoria estatística e a aplicação dos métodos se sustenta no casamento entre o rigor matemático e a modernidade computacional. Qualquer das partes que for negligenciada ocasionará problemas ou limitações importantes.

Ademais, é válido ressaltar a seção extra presente no material, desenvolvida com o intuito de explicar a estrutura dos livros digitais, como esse, usando o *Quarto*. Ainda que muitos considerem adequado o uso do *Bookdown* para este tipo de trabalho (o que é totalmente válido, dadas as especificidades do Rmarkdown), o *Quarto* se mostrou uma espécie de sucessor natural muito adequado, apresentando ferramentas modernas e extremamente úteis para detalhes de *layout*, especialmente tratando de documentos compilados em *HTML*.

Por fim, deixo aqui minha gratidão à Universidade Federal de Juiz de Fora (UFJF) pelo ambiente acadêmico. Agradeço, especialmente, ao meu orientador, Professor Doutor Lupércio França Bessegato, pela oportunidade e pelo acompanhamento técnico que possibilitaram o desenvolvimento deste trabalho.

1 Introdução

Na prática, a estatística contemporânea vai muito além do domínio teórico dos fundamentos, da modelagem probabilística e da inferência. A sociedade atual convive com uma realidade em que os dados são o produto mais valioso existente, além simplesmente de fontes de informação. Dessa forma, observa-se nos últimos anos o surgimento de conceitos como o *Big Data*, para descrever o fenômeno da quantidade massiva de dados que alteram significativamente o fluxo de trabalho do analista de dados, exigindo deste profissional, muitas vezes, muito além da capacidade de analisar dados. É necessário ter aptidão para manusear os dados apropriadamente, buscá-los em diferentes fontes, criar filtros específicos relativos às necessidades de um determinado problema, entre outras habilidades.

Neste contexto, este material foi elaborado com o intuito de introduzir a linguagem *Python* no arsenal analítico de estudantes e profissionais de Estatística, demonstrando sua viabilidade e potência para a análise de dados e a automação de rotinas de trabalho.

1.1 A importância do Python na Estatística

Tradicionalmente, a linguagem *R* ocupa um lugar importante na formação do estatístico, por ser uma linguagem vetorizada e já desenvolvida para o trabalho com dados. Porém, apesar do poder do *R* e seus pacotes nos trabalhos de modelagem de dados e no uso de testes clássicos (paramétricos e não-paramétricos), o *Python* consolidou-se como uma linguagem extremamente versátil, que permite aplicações robustas (como as de *Machine Learning*, por exemplo) e a integração de sistemas.

Em termos de aprendizado, o *Python* é facilmente assimilado por quem já teve contato com algoritmos, por conta de sua similaridade com linguagens compiladas, como no caso do *C++* (exigindo apenas adaptações sintáticas), e com linguagens interpretadas, como o *R*.

Na prática, o *Python* apresenta sua maior vantagem ao analisarmos sua quantidade de usuários e, consequentemente, sua quantidade de bibliotecas desenvolvidas para diferentes aplicações. No contexto da Estatística, destacamos o *pandas*, para estruturação de dados tabulares, e o *numpy*, para computação numérica de alta performance. O domínio dessas ferramentas permite ao analista processar grandes volumes de dados com eficiência, além de explorar diferentes possibilidades, como a de romper com a dependência de bancos de dados pré-estruturados através da automatização da coleta de dados via *internet*.

1.2 O escopo deste material

Este material tem um teor introdutório e prático. Ele não tem a intenção de esgotar as funcionalidades do *Python* (que seria, inclusive, uma tarefa praticamente impossível), tampouco de aprofundar-se nas teorias estatísticas que fundamentam nossas análises ou na teoria computacional

que fundamenta e explica as especificidades da linguagem. O objetivo central é ser uma ponte rápida para profissionais de Estatística compreenderem a linguagem e o ecossistema, e como podem se beneficiar deles.

Neste âmbito, a abordagem adotada se resume em execução de códigos e análise de resultados práticos, de forma que o conhecimento seja construído sequencialmente, como se segue:

- Capítulo 2 - Configurando o Ambiente de Trabalho: Apresenta as instruções necessárias para preparar o ambiente de desenvolvimento, abordando a integração do *Python* dentro do *RStudio* (via pacote *reticulate*) e o gerenciamento de bibliotecas.
- Capítulo 3 - Fundamentos do *Python* para Análise de Dados: Revisa as operações básicas, tipos de dados e a criação de funções, estabelecendo a base sintática da linguagem.
- Capítulo 4 - Manipulação e Análise Exploratória de Dados: Introduz a biblioteca *pandas* e a criação de visualizações, utilizando conjuntos de dados clássicos (como *Titanic* e *Iris*) para demonstrar o tratamento e a exploração de informações categóricas e numéricas.
- Capítulo 5 - Computação Numérica com *NumPy*: Foca no uso de arrays multidimensionais e operações vetorizadas, ilustrando seu poder computacional através de simulações estatísticas.
- Capítulo 6 - Coleta de Dados via *Web Scraping*: Demonstra como construir bases de dados próprias automatizando a extração de informações de páginas da internet, sempre pautado por boas práticas e ética na raspagem de dados.
- Capítulo 7 - Estrutura de Livros Digitais (via *Quarto*): Explica brevemente a estrutura de arquivos e códigos para o desenvolvimento de livros digitais, utilizando o *Rmarkdown* para os códigos e textos e a ferramenta *Quarto* para a compilação.

Espera-se que, ao final deste conteúdo, o estudante ou profissional de Estatística sinta-se confortável para importar, manipular e coletar dados em *Python*, além de estar apto a construir livros digitais que podem ser ferramentas ainda mais robustas que os relatórios padrão, tendo base sólida para estudos posteriores mais aprofundados nestas ferramentas e suas possibilidades.

2 Configurando o Ambiente de Trabalho

Antes de iniciarmos a manipulação de dados e as análises, é fundamental garantir que o ambiente computacional esteja configurado corretamente. Para a maioria dos estudantes e profissionais de Estatística, o *RStudio* é o ambiente de desenvolvimento integrado (IDE) mais familiar. A boa notícia é que o *RStudio*, especialmente quando aliado ao *Quarto*, oferece um suporte excelente para a execução de códigos em *Python*.

2.1 O Ambiente *Python* e o pacote *reticulate*

Em alguns casos, especialmente na primeira execução de um script *Python* dentro do *RStudio*, o ambiente pode não estar perfeitamente configurado. Isso ocorre porque o *RStudio* utiliza um pacote chamado *reticulate* para fazer a ponte e integrar o *Python* ao ecossistema do *R*.

O *reticulate* pode acabar criando, de forma automática, um ambiente virtual *Python* próprio, isolado daquele que você eventualmente já tenha instalado em seu sistema operacional. Por conta desse isolamento, esse novo ambiente nem sempre possui todas as bibliotecas necessárias para a análise de dados já pré-instaladas.

2.2 Gerenciamento de Bibliotecas com o *pip*

Caso apareçam mensagens de erro indicando que bibliotecas fundamentais como *pandas*, *jupyter* ou outras não foram encontradas, elas podem ser facilmente adicionadas utilizando o *pip*. O *pip* é o gerenciador de pacotes padrão do *Python*, responsável por baixar, instalar e atualizar bibliotecas externas.

Para realizar a instalação via **Terminal** (aba **Terminal** no *RStudio*, e não no Console), basta executar os seguintes comandos:

```
python -m pip install pandas numpy seaborn matplotlib
python -m pip install jupyter
```

2.3 Instalação Direta em Blocos de Código

Em documentos interativos (como notebooks *Jupyter* ou arquivos *.qmd* do *Quarto*), também é possível executar comandos de terminal diretamente de dentro das células de código (*code chunks*). Para isso, utiliza-se o caractere de exclamação (!) no início da linha.

Por exemplo, o comando a seguir instala a biblioteca *pandas* diretamente a partir do ambiente de execução do documento, sem a necessidade de abrir o terminal do sistema:

```
!python -m pip install pandas
```

Dica importante: Caso o *RStudio* continue apresentando problemas ou pareça estar utilizando uma instalação do *Python* diferente da esperada, pode ser necessário reiniciar a sessão do *R* (*Session* > *Restart R*) para que o *reticulate* reconheça as novas bibliotecas instaladas. Em geral, após essa configuração inicial e a instalação dos pacotes, o código passará a funcionar normalmente.

3 Fundamentos

Neste capítulo, iremos contextualizar a lógica de algoritmos ao *Python*, explicando brevemente a sua sintaxe essencial. A lógica de programação se mantém a mesma independentemente do ambiente computacional, mas detalhes estruturais como a tipagem de dados, a delimitação de blocos e a indexação variam significativamente de uma linguagem para a outra (Sebesta, 2018).

Enquanto linguagens tipadas estaticamente exigem a declaração prévia do tipo de cada variável (seja ela um número inteiro ou um texto), o *Python* utiliza a tipagem dinâmica, inferindo o tipo do dado no momento de sua execução (Lutz, 2013). Além disso, diferentemente de linguagens que utilizam chaves para agrupar comandos, o *Python* adota a indentação obrigatória como regra sintática, o que impõe a elaboração de códigos padronizados e altamente legíveis. Outro fator de atenção para os analistas e desenvolvedores é a indexação: ao contrário do *R* (que é também uma linguagem amplamente utilizada no contexto de análise de dados e Estatística), onde o primeiro elemento de uma estrutura de dados ocupa o índice 1, o *Python* adota a indexação baseada em zero (0-based), característica compartilhada com linguagens como *C* e o *C++*.

3.1 Filosofia, Sintaxe e Tipagem da Linguagem

Grande parte das linguagens de programação surgem (ou evoluem) focadas em otimização de performance para a máquina. Porém, esse não é o caso do *Python*, que foi concebido a partir de uma filosofia muito bem fundamentada e orientada ao desenvolvedor. O princípio norteador da linguagem é o de que um código é lido muito mais vezes do que ele é escrito. Portanto, a clareza e a legibilidade devem ser prioridades absolutas.

Essa filosofia está muito bem documentada no Zen do *Python* (PEP 20), um conjunto de princípios que guiam as melhores práticas na linguagem, destacando máximas como “o simples é melhor que o complexo” e “a legibilidade conta” (Peters, 2004).

Para garantir essa legibilidade de forma estrutural, a sintaxe do *Python* elimina o uso de marcadores visuais excessivos, como as chaves `{}` ou as expressões `begin/end` para delimitar blocos de código. Em seu lugar, a linguagem adota a indentação obrigatória. O recuo da margem esquerda não é meramente uma questão de estilo, mas uma regra sintática rígida documentada na PEP 8 (Van Rossum; Warsaw; Coghlan, 2001). Seu objetivo é forçar o programador a organizar, hierarquicamente, as operações, resultando em um código mais limpo.

Além da sintaxe visual, o *Python* caracteriza-se por uma tipagem dinâmica e forte. Dinâmica porque não exige a declaração prévia do tipo da variável - a própria linguagem infere se o dado é numérico ou textual no momento da atribuição, além de permitir mudanças de valor e de tipo durante o processo de desenvolvimento. Forte porque, uma vez inferido o tipo, o *Python* não realiza conversões implícitas que possam gerar comportamentos inesperados, garantindo a segurança e a consistência das operações.

3.2 Estruturas de Dados Nativas

Conhecer as estruturas de dados nativas do *Python* é essencial para armazenar e organizar informações. As principais coleções embutidas na linguagem são as listas (`list`), as tuplas (`tuple`), os dicionários (`dict`) e os conjuntos (`set`).

No contexto da manipulação de dados e da programação geral, a **lista** destaca-se como a estrutura mais versátil e frequentemente utilizada. Diferentemente dos vetores atômicos encontrados no *R* ou dos *arrays* estáticos comuns no *C++*, as listas em *Python* são coleções heterogêneas e dinâmicas. Isso significa que uma única lista pode armazenar, de forma simultânea, números inteiros, valores reais, textos ou até mesmo outras listas em seu interior (Lutz, 2013).

Além da heterogeneidade, as listas são estruturas **mutáveis**. Após sua criação, o programador ou analista pode alterar seu conteúdo em tempo de execução sem a necessidade de realocar a estrutura inteira na memória. É possível modificar valores existentes através de seus índices, adicionar novos elementos ao final da coleção (utilizando o método `append()`) ou remover o último item inserido (através do método `pop()`) (Python Software Foundation, 2026a). Essa flexibilidade de manipulação direta torna as listas uma base poderosa para o tratamento inicial de dados antes de avançarmos para o uso de bibliotecas.

3.3 Estruturas de Controle de Fluxo

A espinha dorsal de qualquer linguagem de programação é a capacidade de direcionar a execução dos algoritmos, através de estruturas de controle de fluxo. Destacam-se, nesse contexto, as condições lógicas e os laços de repetição, que são amplamente conhecidos e utilizados por quem trabalha com códigos, seja qual for o contexto. Em *Python*, as estruturas de controle de fluxo seguem a filosofia da linguagem - citada neste capítulo - sendo projetadas para serem diretas e facilmente legíveis, se comparadas a outras linguagens de programação.

As estruturas condicionais utilizam as palavras-chave `if`, `elif` (uma abreviação de `else if`) e `else`. A condição avaliada não exige o uso de parênteses, e o bloco de código subordinado é demarcado de forma simples e intuitiva: através da inserção de dois pontos (`:`) ao final da instrução, seguida da já mencionada indentação obrigatória.

No que diz respeito às estruturas de repetição, o *Python* oferece os laços `while` e `for`. O laço `while` opera de maneira tradicional, executando um bloco de código enquanto uma condição permanecer verdadeira. Contudo, o laço `for` apresenta uma diferença fundamental e vantajosa em relação a linguagens compiladas clássicas. Em vez de iterar primariamente sobre uma progressão aritmética de índices, o `for` em *Python* atua como um iterador direto sobre os itens de qualquer sequência, como uma lista ou uma tupla (Python Software Foundation, 2026a).

No âmbito da análise de dados, por exemplo, essa característica permite que o analista percorra os dados de uma coleção diretamente (por exemplo, lendo cada observação de uma variável) sem a necessidade de gerenciar variáveis contadoras, o que reduz significativamente a chance de erros lógicos. Caso uma iteração numérica tradicional seja estritamente necessária, a linguagem fornece uma função embutida `range()`, capaz de gerar sequências numéricas de forma eficiente. É importante destacar que a função `range()` não armazena a sequência inteira na memória de uma só vez; ela calcula os valores sob demanda a cada passo da iteração. Consequentemente, sua complexidade de

espaço (consumo de memória) é constante, $O(1)$ ¹, independentemente de a sequência possuir dez ou dez milhões de elementos (Python Software Foundation, 2026b). Isso evita gargalos de performance e torna o código otimizado.

3.4 Funções e Ferramentas Funcionais

O processo de desenvolvimento de um algoritmo não apresenta **obrigatoriedade** de práticas que vão além de uma lógica concisa e do respeito à sintaxe da linguagem. Porém, a escalabilidade de um *software* ou a estrutura de uma análise de dados, por exemplo, exigem boas práticas que são fundamentadas no bom uso das funções (métodos), antes de partir para estruturas mais complexas. No *Python*, a definição de uma função é realizada através da palavra-chave **def**, seguida do nome da função e de seus respectivos parâmetros. Assim como nas funções prontas do *Python*, as funções customizadas também exigem, por padrão da sintaxe, os parênteses logo após o nome da função. Portanto, os parâmetros são criados dentro dos parênteses, e o valor de saída da operação, ao final da função, é especificado pelo comando **return**.

Contudo, como citado neste capítulo, o *Python* não se limita ao paradigma procedural; ele incorpora elementos poderosos do paradigma de programação funcional. Para evitar a escrita de laços **for** muito extensos ao transformar bases de dados, a linguagem fornece funções de alta ordem (*higher-order functions*), sendo as mais notáveis o **map()** e o **filter()** (Python Software Foundation, 2026c).

Para aqueles que estão habituados ao uso da linguagem *R*, muito usada na Estatística e na análise de dados, essa mecânica é conceitualmente idêntica ao uso de funções **apply()** (como o **lapply()** e o **sapply()**). O objetivo é o mesmo: permitir um processamento de dados mais rápido, conciso e com menos linhas de código. O **map()** aplica uma mesma transformação a todos os itens de uma coleção de forma simultânea, enquanto o **filter()** extrai e retorna apenas os elementos que satisfazem uma determinada condição lógica (bem próximo do que faz a função **which()**, no *R*).

Essas ferramentas são ainda mais ágeis quando combinadas com as expressões **lambda**. Uma função **lambda** é uma função anônima — ou seja, uma função que não precisa ser formalmente nomeada com **def** e que é criada em uma única linha de código, sendo ideal para operações rápidas ou descartáveis que serão usadas apenas uma vez dentro de um **map** ou **filter** (Lutz, 2013).

3.5 Aplicação Prática: Manipulação de Listas

No script a seguir, consolidamos todos os fundamentos apresentados neste capítulo. O exemplo ilustra desde a manipulação direta de uma lista utilizando métodos nativos (como o **append()** e o **pop()**), até o uso avançado das funções **map()** e **filter()** associadas a funções anônimas (**lambda**). Por fim, demonstramos a aplicação de uma função customizada, desenvolvida com **def()**, para o cálculo de uma média simples.

¹A notação $O(1)$ faz parte da chamada “Notação *Big O*” (*Big O Notation*), utilizada na Ciência da Computação para descrever matematicamente a eficiência e a escalabilidade de um algoritmo. Uma complexidade de espaço $O(1)$, ou constante, indica que a quantidade de memória exigida para a execução da tarefa permanece exatamente a mesma, não importando se a entrada possui dez ou dez milhões de elementos.

Nota-se, também, o uso das chamadas *f-strings* (`f"texto {variável}"`) dentro da função `print()`. Esse é o método mais moderno e recomendado no *Python* para interpretar textos e variáveis na mesma linha, gerando saídas de telas mais limpas.

```
# ---- Definição de uma função customizada ----
def media(lista):
    soma = sum(lista)
    quantidade = len(lista)
    return soma / quantidade

# ---- Criação e manipulação direta da lista ----
valores = [10, 20, 30, 40]
print(f"Lista original: {valores}")

# Uso do append()
valores.append(50)
print(f"Após append (adicionar 50): {valores}")

# Uso do pop()
valores.pop()
print(f"Após pop (remover último elemento): {valores}")

# ---- Uso de ferramentas funcionais (map, filter e lambda) ----

# Uso do map() com lambda
valores_dobrados = list(map(lambda x: x * 2, valores))
print(f"Valores dobrados com map: {valores_dobrados}")

# Uso do filter() com lambda
valores_filtrados = list(filter(lambda x: x > 20, valores))
print(f"Valores > 20 com filter: {valores_filtrados}")

# ---- Aplicação da função customizada ----
resultado = media(valores)
print(f"Média final dos valores originais: {resultado}")
```

```
Lista original: [10, 20, 30, 40]
Após append (adicionar 50): [10, 20, 30, 40, 50]
Após pop (remover último elemento): [10, 20, 30, 40]
Valores dobrados com map: [20, 40, 60, 80]
Valores > 20 com filter: [30, 40]
Média final dos valores originais: 25.0
```

4 Análise Exploratória e Manipulação de Dados em *Python*

Chegamos no ponto-chave de qualquer análise de dados. Antes da modelagem, da inferência e da extração de resultados estatísticos, é a etapa exploratória que contextualiza o analista. Sem a compreensão prévia do cenário, qualquer procedimento será puramente matemático: exato em termos teóricos, porém vazio de informação aplicável ou interpretável. A função da Estatística é justamente atribuir significado aos dados, contando a sua história por meio de metodologias robustas e bem fundamentadas.

Dessa maneira, é preciso compreender a essência das informações em mãos. É através da Análise Exploratória de Dados (AED) que esse processo ganha vida e, no ecossistema *Python*, ele é conduzido primordialmente com o suporte de bibliotecas especializadas na manipulação e visualização de estruturas tabulares[^][Uma estrutura tabular refere-se à organização bidimensional dos dados em formato de tabela. Na terminologia estatística, equivale a uma matriz de dados, onde cada linha representa uma observação (ou registro) e cada coluna representa uma variável (ou atributo) de interesse.

4.1 A Biblioteca *Pandas* e as Estruturas de Dados

Durante muito tempo, a manipulação de dados tabulares não era uma tarefa trivial em *Python*, sendo outras linguagens — como o *R* — preferidas para análises estatísticas. Para suprir a necessidade de ferramentas analíticas flexíveis e de alta performance, o pesquisador Wes McKinney iniciou, em 2008, o desenvolvimento da biblioteca **pandas**. Hoje, ela se consolidou como o padrão ouro da indústria para a limpeza, transformação e análise de dados em *Python* (McKinney, 2022). O ecossistema do **pandas** é construído sobre duas estruturas de dados fundamentais: a **Series** e o **DataFrame**.

Uma **Series** pode ser definida como um vetor unidimensional (semelhante a uma coluna de uma tabela) capaz de armazenar qualquer tipo de dado, acompanhada por um vetor de rótulos chamado de índice (*index*).

Já o **DataFrame** — a estrutura principal da biblioteca — é um objeto bidimensional tabular. Para o estatístico, ele pode ser compreendido como a equivalência exata de um **data.frame** na linguagem *R* ou de uma planilha eletrônica estruturada. Matematicamente, um **DataFrame** é essencialmente um dicionário onde cada coluna é uma **Series** que compartilha o mesmo índice posicional (McKinney, 2022).

Apesar da semelhança teórica com o *R*, a manipulação dessas estruturas apresenta peculiaridades sintáticas importantes. Enquanto no *R* a indexação se inicia em 1, a do **pandas** segue a regra do

Python (baseada em zero), e a seleção explícita de dados é intermediada por métodos específicos da biblioteca, como os localizadores `.loc` (para rótulos) e `.iloc` (para posições inteiras). A partir dessa estrutura bidimensional, o analista ganha acesso imediato a métodos otimizados para a seleção de colunas, filtragem de linhas, criação de novas variáveis e tratamento de valores ausentes.

4.2 Importação e Tratamento de Dados (*Data Wrangling*)

Antes que qualquer inferência estatística ou visualização seja possível, os dados brutos precisam ser carregados para a memória e rigorosamente limpos. Na comunidade analítica, esse processo de estruturação e higienização é frequentemente chamado de *Data Wrangling*.

O primeiro passo é a importação dos dados. O **pandas** oferece diversas opções para a leitura de dados, sendo o método `pd.read_csv()` o mais trivial deles. Ao contrário das funções básicas de leitura em outras linguagens, o **pandas** possui um motor de inferência de tipos altamente otimizado, lidando de forma automática com a alocação de memória para variáveis numéricas, textuais ou séries temporais no momento da leitura (VanderPlas, 2016).

Uma vez importada a matriz de dados, é muito comum nos depararmos com células de valores ausentes (*missing values*). No *R*, estes valores são representados como `NA`, enquanto no *Python* eles são representados como `NaN`. Ambos representam a mesma coisa, mudando apenas a forma de apresentação. É evidente que o tratamento de valores ausentes é um tópico de muita discussão teórica na Estatística, já que alguns métodos, como o de preenchê-los com a média, por exemplo, podem nos levar a subestimar a variância em alguns casos, o que introduziria um viés na análise dos dados. Portanto, este tipo de tratamento deve sempre ser realizado com atenção considerando que ele não depende apenas da aplicação de funções computacionais.

Neste contexto, dado que o analista fez, previamente, uma análise robusta que garanta a viabilidade de algum método de preenchimento de valores ausentes, o **pandas** oferece ótimas ferramentas práticas para a aplicação dos métodos: a função `dropna()` permite a exclusão direta das observações ou variáveis incompletas, enquanto a `fillna()` viabiliza técnicas de imputação, como o preenchimento de falhas utilizando a média, a mediana ou o último valor válido observado da respectiva série.

Outro pilar crítico do *Data Wrangling* é o cruzamento de bases relacionais. Projetos reais raramente dependem de um único arquivo isolado; é frequente a necessidade de unir tabelas distintas por meio de variáveis-chave, utilizando a função `pd.merge()`. Nesta fase, a integridade lógica da base final deve ser a prioridade. Como regra fundamental de validação de dados em junções, caso ocorram discrepâncias ou identificadores duplicados ao combinar duas bases, os valores oriundos da primeira tabela (considerada a base primária ou de maior confiabilidade) devem sempre prevalecer. No *Python*, esse controle lógico rigoroso é facilitado pela arquitetura de *joins* do **pandas** e por métodos de alinhamento de índices, garantindo que o tratamento de duplicidades ou conflitos não comprometa a informação original mais segura.

4.3 Visualização dos Dados

A qualidade de visualização dos dados é essencial na Estatística. É através dos recursos gráficos que podemos identificar tendências, anomalias e até mesmo distribuições. A visualização constitui a fase inicial de qualquer análise descritiva dos dados, mas não se limita a apenas isso. É comum,

por exemplo, utilizar o QQ-plot para avaliar a normalidade dos resíduos de um modelo, de forma a dar suporte aos testes não-paramétricos que podem levar o analista à conclusões equivocadas em alguns casos, se utilizados de forma isolada. É justamente por este motivo que a utilização de mais de um recurso é sempre recomendada para testar pressuposições teóricas importantes.

No âmbito visual, o *Python* conta com bibliotecas muito poderosas. São elas, principalmente, o `matplotlib` e o `Seaborn`. O `matplotlib` atua como o motor estrutural da linguagem. Ele é responsável por instanciar e desenhar os elementos fundamentais da interface gráfica, como a figura (a “tela” em branco), os eixos cartesianos, as marcações e as escalas (Hunter, 2007). Por ser uma ferramenta de baixo nível (termo que, na computação, refere-se à menor distância conceitual entre o código e a máquina, indicando a necessidade de especificar instruções mais minuciosas, e não uma avaliação sobre a qualidade da ferramenta) e extremamente detalhista, ele exige uma quantidade considerável de código para gerar representações esteticamente polidas, assemelhando-se aos pacotes base de gráficos do R (como a função `plot()`).

Para suprir a necessidade de gráficos estatísticos mais expressivos e com menos linhas de código, surgiu o `seaborn`. Construída nativamente sobre o `matplotlib`, essa biblioteca funciona como uma interface declarativa de alto nível, sendo perfeitamente integrada aos objetos da biblioteca `pandas` (Waskom, 2021).

O grande diferencial do `seaborn` para o analista de dados é a abstração de complexidades estruturais somada à automatização de agregações estatísticas. Em muitas de suas funções, a biblioteca calcula e exibe intervalos de confiança de forma automática ao agregar dados. Além disso, o `seaborn` traz paletas de cores nativas fundamentadas na teoria da percepção visual, garantindo que as distinções categóricas e as intensidades contínuas sejam compreendidas de maneira intuitiva e sem vieses de leitura.

4.4 Estudos de Caso Práticos

Para consolidar os conceitos de *Data Wrangling* e visualização, aplicaremos as ferramentas do `pandas` e do `seaborn` em dois conjuntos de dados clássicos na literatura estatística e no aprendizado de máquina: a base `Titanic` (focada em dados categóricos e taxas de sobrevivência) e a base `Iris` (focada em dados contínuos e distribuições dimensionais).

Para facilitar a reprodutibilidade dos códigos a seguir, utilizaremos as versões dessas bases que já vêm nativamente embutidas na biblioteca `seaborn`.

4.4.1 Análise de Dados Categóricos: *Titanic*

O conjunto de dados do naufrágio do `Titanic` é ideal para a exploração de tabelas de contingência e distribuições de frequência. No *Python*, a função `pd.crosstab()` do `pandas` atua de forma muito semelhante à clássica função `table()` do R, permitindo o cruzamento rápido entre variáveis qualitativas.

No script abaixo, importamos as bibliotecas necessárias, carregamos os dados, removemos rapidamente os valores ausentes de uma coluna de interesse estrutural e geramos a tabela de contingência cruzando a classe do passageiro com o status de sobrevivência. Em seguida, utilizamos o `seaborn` para criar um gráfico de barras.

```

import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Carregando a base de dados
titanic = sns.load_dataset('titanic')

# Tratamento rápido (Data Wrangling): Removendo linhas sem informação de embarque
titanic_limpo = titanic.dropna(subset=['embark_town'])

# Tabela de contingência (Classe vs Sobrevivência)
tabela_sobrevivencia = pd.crosstab(
    titanic_limpo['class'],
    titanic_limpo['survived'],
    rownames=['Classe'],
    colnames=['Sobreviveu?']
)
print("Tabela de Contingência - Titanic:\n")
print(tabela_sobrevivencia)

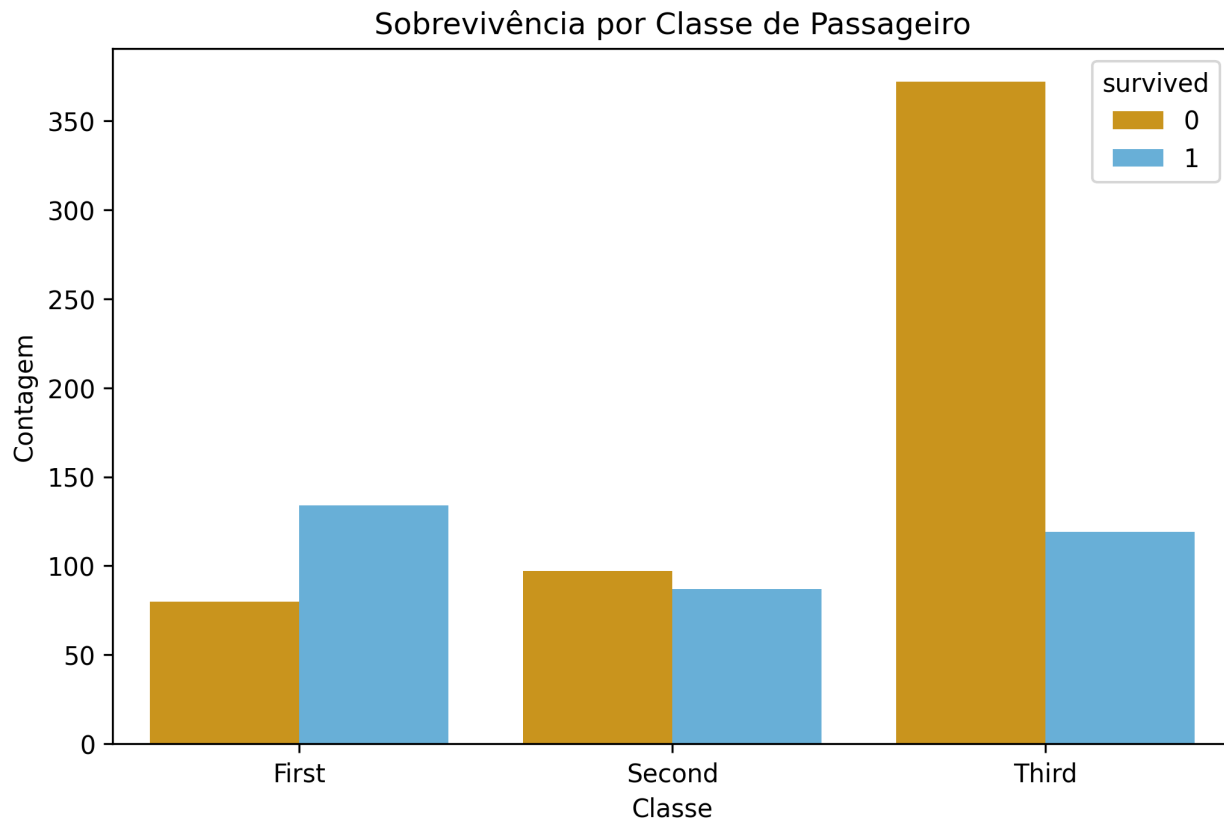
# Configurando e gerando o gráfico com Seaborn
plt.figure(figsize=(8, 5))
sns.countplot(
    data=titanic_limpo,
    x='class',
    hue='survived',
    palette=["#E69F00", "#56B4E9", "#CC79A7"]
)
plt.title("Sobrevivência por Classe de Passageiro")
plt.xlabel("Classe")
plt.ylabel("Contagem")
plt.show()

```

Tabela de Contingência - Titanic:

Sobreviveu?	0	1
Classe		
First	80	134
Second	97	87
Third	372	119

C:\Users\gabri\AppData\Local\Temp\ipykernel_13920\2615282862.py:23: UserWarning: The palette list is too long to be used with the 'hue' parameter in `sns.countplot()`



4.4.2 Análise de Dados Contínuos: *Iris*

A base *Iris*, introduzida pelo estatístico *Ronald Fisher* em 1936, é o exemplo definitivo para o estudo de distribuições contínuas e análise de correlação entre múltiplas variáveis numéricas (como comprimento e largura de pétalas e sépalas).

Antes de partirmos para a visualização, a extração de métricas de posição e dispersão é o passo primário para a compreensão de variáveis contínuas. No *R*, essa tarefa é comumente realizada através da função `summary()`. No *Python*, a biblioteca `pandas` fornece o método `.describe()`, que calcula instantaneamente a contagem, a média, o desvio padrão e os quartis de todas as colunas numéricas do *DataFrame*.

Enquanto as métricas descritivas resumem os dados, a exploração de dados contínuos exige uma abordagem visual para a identificação de padrões lineares ou de agrupamentos (*clusters*). O gráfico de dispersão (*scatterplot*) é a ferramenta analítica primária para esse fim. O `seaborn` permite estratificar esses pontos facilmente através do parâmetro `hue`, colorindo as observações com base em uma variável categórica, como a espécie da flor.

```
# Carregando a base de dados
iris = sns.load_dataset('iris')

# Extração das métricas descritivas (equivalente ao summary do R)
print("Resumo Estatístico da Base Iris:\n")
```

```

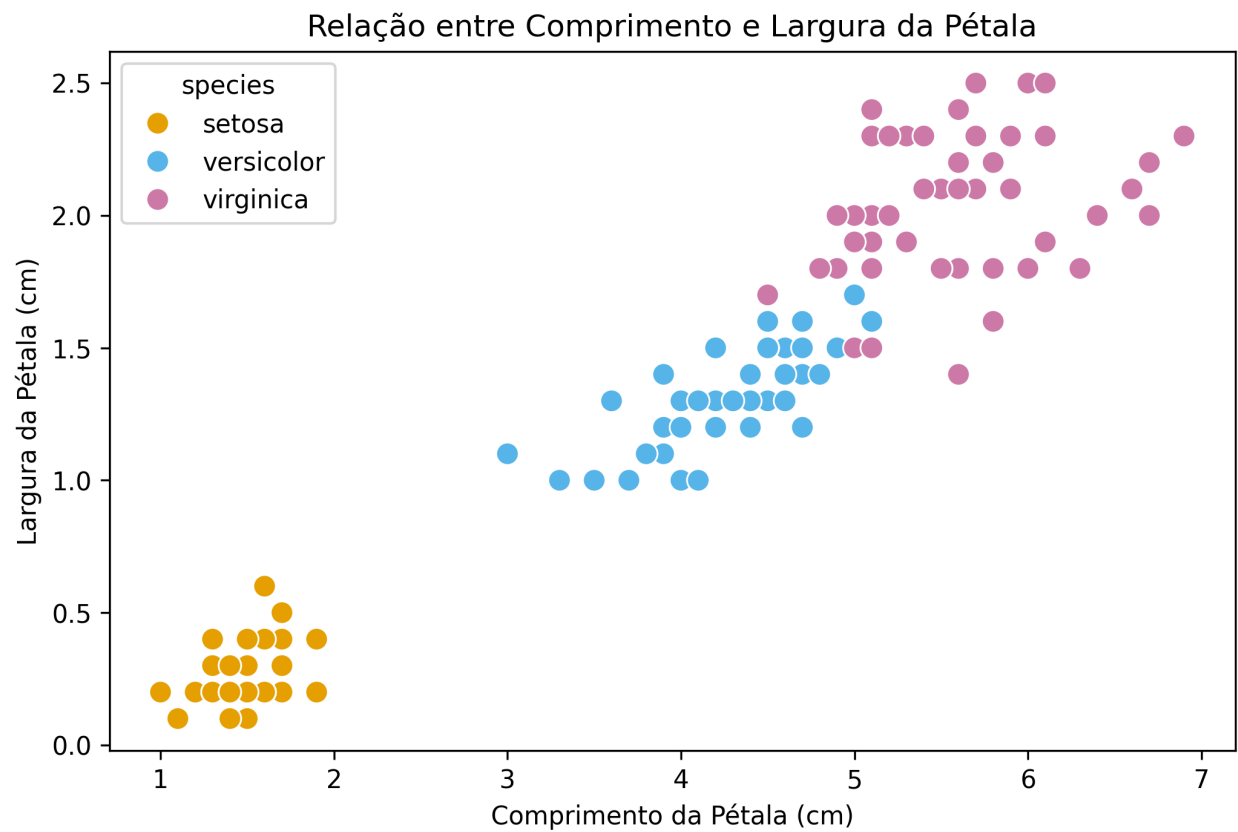
print(iris.describe())

# Configurando e gerando o gráfico de dispersão com Seaborn
plt.figure(figsize=(8, 5))
sns.scatterplot(
    data=iris,
    x='petal_length',
    y='petal_width',
    hue='species',
    palette=["#E69F00", "#56B4E9", "#CC79A7"],
    s=80 # Tamanho dos pontos
)
plt.title("Relação entre Comprimento e Largura da Pétala")
plt.xlabel("Comprimento da Pétala (cm)")
plt.ylabel("Largura da Pétala (cm)")
plt.show()

```

Resumo Estatístico da Base Iris:

	sepal_length	sepal_width	petal_length	petal_width
count	150.000000	150.000000	150.000000	150.000000
mean	5.843333	3.057333	3.758000	1.199333
std	0.828066	0.435866	1.765298	0.762238
min	4.300000	2.000000	1.000000	0.100000
25%	5.100000	2.800000	1.600000	0.300000
50%	5.800000	3.000000	4.350000	1.300000
75%	6.400000	3.300000	5.100000	1.800000
max	7.900000	4.400000	6.900000	2.500000



5 Computação Numérica com NumPy

Apesar da versatilidade e da sintaxe amigável, podemos encontrar problema ao trabalhar com computação matemática em *Python*. Isso ocorre porque o *Python* não foi concebido originalmente para o trabalho científico, ao contrário do *R* por exemplo, que já foi desenvolvida de forma vetorizada, voltada para trabalhar com Estatística.

É para preencher essa lacuna que surge a biblioteca **numpy** (*Numerical Python*). Ela atua como o alicerce do ecossistema de dados da linguagem, sendo, inclusive, a base matemática oculta que atua por trás do **pandas**, do **matplotlib** e de bibliotecas de aprendizado de máquina.

5.1 A Revolução Vetorial e o Objeto

Como vimos, a principal estrutura de dados do *Python* para armazenar vetores de dados é a lista (**list**). As listas são extremamente flexíveis, permitem armazenar dados de diferentes tipos nativamente e, até mesmo, armazenar outras listas dentro de uma lista. Porém, do ponto de vista computacional, essa maleabilidade tem um custo elevado: as listas não armazenam os dados em si de forma contínua na memória, mas sim ponteiros para onde as informações estão. Como é de se imaginar, isso torna o processamento de grandes bases de dados extremamente custoso computacionalmente falando, já que as iterações estariam sendo feitas sobre os ponteiros.

A revolução trazida pelo **numpy** baseia-se na introdução de um novo objeto fundamental: o *array* multidimensional, conhecido pela classe **ndarray** (*n-dimensional array*) (Harris et al., 2020). Diferentemente das listas, um **ndarray** exige tipagem estática, o que significa que todos os elementos em seu interior devem ser estritamente do mesmo tipo (geralmente numérico). Essa homogeneidade permite que os dados sejam alocados na memória de forma adjacente, fazendo com que a eficiência dos cálculos se aproxime bastante a de linguagens compiladas, como o *C++*, por exemplo.

Para o trabalho estatístico, a maior vantagem do **ndarray** é permitir a vetorização. Operações matemáticas aplicadas a um *array* são executadas em toda a estrutura simultaneamente, eliminando por completo a necessidade de escrever laços de repetição manuais (*for* ou *while*).

O bloco de código a seguir ilustra a diferença na sintaxe e no comportamento ao realizarmos uma soma elemento a elemento entre dois vetores numéricos usando o *Python* puro e o **numpy**:

```
import numpy as np

# Soma elemento a elemento com Python puro (Listas)
lista_a = [1, 2, 3, 4]
lista_b = [10, 20, 30, 40]
```

```
# Exige conhecimento de listas e iteração explícita com a função zip
soma_listas = [a + b for a, b in zip(lista_a, lista_b)]
print(f"Soma com listas nativas: {soma_listas}")

# Soma elemento a elemento com NumPy (ndarrays)
array_a = np.array([1, 2, 3, 4])
array_b = np.array([10, 20, 30, 40])

# Operação vetorizada direta, parecido com o R
soma_arrays = array_a + array_b
print(f"Soma com arrays do NumPy: {soma_arrays}")
```

```
Soma com listas nativas: [11, 22, 33, 44]
Soma com arrays do NumPy: [11 22 33 44]
```

5.2 Criação e Formatação de Matrizes

Na prática, raramente digitamos manualmente os valores de uma matriz. Neste contexto, o **numpy** oferece diversas funções geradoras otimizadas para criar vetores e matrizes (McKinney, 2022).

As funções `np.zeros()` e `np.ones()` criam *arrays* preenchidos exclusivamente por zeros ou uns, respectivamente, bastando informar a dimensão desejada. Para criar sequências numéricas, o `np.arange()` funciona de maneira análoga à função `seq()` do *R* ou `range()` do *Python*, gerando valores com base em um limite inicial, um limite final e um passo. Já o `np.linspace()` é extremamente útil na criação de pontos para distribuições ou gráficos, pois gera um número exato de valores linearmente espaçados dentro de um intervalo fechado.

Além da criação, a manipulação da dimensionalidade é uma tarefa constante preparação dos dados, com a qual todo analista tem que lidar. Todo *array* do **numpy** possui o atributo `.shape`, que retorna uma tupla com o tamanho de cada dimensão, e o atributo `.ndim`, que indica o número total de dimensões (1 para vetores, 2 para matrizes, etc.).

Caso seja necessário alterar a estrutura de um conjunto de dados sem modificar seus valores numéricos originais — como transformar um vetor unidimensional de 12 elementos em uma matriz bidimensional de 3 linhas e 4 colunas — o método `.reshape()` realiza essa transposição instantaneamente.

O script a seguir demonstra essas ferramentas em ação:

```
# Geração de matrizes básicas
# O parâmetro é uma tupla indicando (linhas, colunas)
matriz_zeros = np.zeros((2, 3))
print(f"Matriz de Zeros:\n{matriz_zeros}\n")

# Sequências numéricas
vetor_sequencia = np.arange(0, 10, 2) # De 0 a 10 (exclusivo), passo 2
print(f"Sequência com arange: {vetor_sequencia}\n")
```

```

vetor_linear = np.linspace(0, 1, 5) # 5 valores uniformemente espaçados entre 0 e 1
print(f"Sequência com linspace: {vetor_linear}\n")

# Manipulação de dimensionalidade
vetor_base = np.arange(1, 13)
print(f"Vetor Original (1D): {vetor_base}")
print(f"Formato: {vetor_base.shape} | Dimensões: {vetor_base.ndim}\n")

# Transformando o vetor 1D em uma matriz 2D (3 linhas, 4 colunas)
matriz_remodelada = vetor_base.reshape(3, 4)
print(f"Matriz Remodelada (2D):\n{matriz_remodelada}")
print(f"Novo Formato: {matriz_remodelada.shape} | Dimensões: {matriz_remodelada.ndim}")

```

Matriz de Zeros:

```

[[0. 0. 0.]
 [0. 0. 0.]]

```

Sequência com arange: [0 2 4 6 8]

Sequência com linspace: [0. 0.25 0.5 0.75 1.]

Vetor Original (1D): [1 2 3 4 5 6 7 8 9 10 11 12]

Formato: (12,) | Dimensões: 1

Matriz Remodelada (2D):

```

[[ 1  2  3  4]
 [ 5  6  7  8]
 [ 9 10 11 12]]

```

Novo Formato: (3, 4) | Dimensões: 2

5.3 Álgebra, *Broadcasting* e Indexação Booleana

A álgebra matricial clássica exige que duas matrizes tenham formatos compatíveis para realizar operações como soma ou multiplicação. Porém, o **numpy** apresenta um recurso computacional extremamente otimizado, chamado de *Broadcasting* (Harris et al., 2020), que aumenta consideravelmente essa flexibilidade operacional. Esse recurso permite que o *Python* alinhe e opere vetores de diferentes dimensões sem a necessidade de duplicar dados na memória.

Um exemplo clássico do *Broadcasting* é quando multiplicamos um *array* por um escalar. O escalar é virtualmente “esticado” ou transmitido por toda a dimensão do array, aplicando a operação a cada elemento individualmente. Isso facilita bastante a padronização de dados estatísticos (como subtrair a média e dividir pelo desvio padrão de um vetor).

Além da álgebra vetorizada, a manipulação de matrizes também requer métodos eficientes de filtragem. Para tal, o *R* oferece um suporte nativo da linguagem, que já é vetorizada em sua base.

No *Python*, é o **numpy** que reproduz essa característica, através da Indexação Booleana (ou Máscara Booleana).

Ao aplicarmos uma condição lógica a um **ndarray** (por exemplo, vetor > 5), a biblioteca não avalia a estrutura inteira como uma única expressão, mas sim elemento a elemento, retornando um novo *array* contendo valores lógicos (*True* ou *False*). Ao passarmos essa “máscara” dentro dos colchetes do *array* original, o **numpy** filtra e retorna exclusivamente os elementos onde a condição for verdadeira.

O código a seguir demonstra o poder combinado do *Broadcasting* e das indexações lógicas:

```
import numpy as np

# Criando um array de base para testes
dados = np.array([12, 15, 22, 9, 31, 18, 4])
print(f"Vetor original: {dados}\n")

# 1. Aplicação de Broadcasting (Escalar sobre Array)
# Multiplicando todos os elementos por 2 e somando 10
dados_transformados = (dados * 2) + 10
print(f"Broadcasting (dados * 2 + 10): {dados_transformados}\n")

# 2. Máscara Booleana (Condição Lógica)
mascara = dados > 15
print(f"Máscara Lógica (dados > 15): {mascara}\n")

# 3. Indexação Booleana (Filtragem direta)
dados_filtrados = dados[dados > 15]
print(f"Vetor filtrado (apenas valores > 15): {dados_filtrados}\n")

# 4. Indexação com múltiplas condições (uso dos operadores & e |)
# Filtrando valores pares E maiores que 10
# Obs: É obrigatório o uso de parênteses em múltiplas condições no numpy
filtro_complexo = dados[(dados > 10) & (dados % 2 == 0)]
print(f"Valores pares maiores que 10: {filtro_complexo}")
```

Vetor original: [12 15 22 9 31 18 4]

Broadcasting (dados * 2 + 10): [34 40 54 28 72 46 18]

Máscara Lógica (dados > 15): [False False True False True True False]

Vetor filtrado (apenas valores > 15): [22 31 18]

Valores pares maiores que 10: [12 22 18]

5.4 Funções Estatísticas e Simulação Aleatória

O último pilar a ser explorado do `numpy`, talvez onde realmente consigamos contextualizar bastante o trabalho do Estatístico usando o *Python*, é a sua capacidade de realizar computação estatística e gerar números pseudoaleatórios. Para cálculos descritivos, os *arrays* contam com métodos nativos altamente otimizados (como `.mean()`, `.sum()`, `.var()` e `.std()`) que operam em frações de segundo, mesmo em vetores com milhões de observações (Harris et al., 2020).

Contudo, para o estatístico, o verdadeiro poder da biblioteca está no submódulo `np.random`. Ele permite a simulação estocástica a partir das mais variadas famílias de distribuições de probabilidade (Normal, Binomial, Poisson, Exponencial, entre outras), sendo a ferramenta base para simulações de Monte Carlo e reamostragem.

Para demonstrar a confiabilidade computacional do `numpy`, realizaremos dois experimentos empíricos clássicos da teoria das probabilidades:

1. A Propriedade da Soma da Distribuição de Poisson

Matematicamente, sabemos que se extrairmos uma amostra aleatória de tamanho n de uma população que segue uma distribuição de Poisson com parâmetro θ , o somatório dos valores dessa amostra não é apenas uma acumulação aleatória; o somatório do parâmetro θ em uma amostra de tamanho n deve ser representado exatamente como $n\theta$. Vamos simular uma amostra grande e verificar o valor empírico contra esse valor teórico esperado.

2. Convergência Assintótica (Teorema Central do Limite)

O *Teorema Central do Limite (TCL)* postula que a distribuição das médias amostrais de variáveis independentes e identicamente distribuídas converge assintoticamente para uma distribuição Normal à medida que o tamanho da amostra (n) cresce, independentemente da distribuição original dos dados. Simularemos milhares de amostras extraídas de uma distribuição estritamente Exponencial (fortemente assimétrica) e utilizaremos o `seaborn` (visto no Capítulo 4) para plotar a densidade das médias, revelando o inconfundível formato de sino.

O script a seguir realiza essas duas simulações:

```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Fixando a semente para garantir a reprodutibilidade do livro
np.random.seed(666)

print("--- 1. Simulação da Distribuição de Poisson ---")
n_poisson = 100000 # Tamanho da amostra
theta = 3.5         # Parâmetro da distribuição

# Gerando o array com a amostra de Poisson
amostra_poisson = np.random.poisson(lam=theta, size=n_poisson)

# Calculando o somatório empírico e o teórico (n * theta)
soma_empirica = np.sum(amostra_poisson)
```

```

soma_teorica = n_poisson * theta

print(f"Soma Empírica do array: {soma_empirica}")
print(f"Valor Teórico Esperado (n * \u03b8): {soma_teorica}\n")

print("--- 2. Teorema Central do Limite (Convergência para Normal) ---")
n_clt = 500          # Tamanho de cada amostra (n grande)
simulacoes = 10000   # Quantidade de amostras geradas
taxa_lambda = 2.0     # Parâmetro da distribuição original (Exponencial)

# Simulando 10.000 médias de amostras de tamanho 500
# Criamos uma matriz (10000, 500) e tiramos a média nas linhas (axis=1)
matriz_exponencial = np.random.exponential(scale=1/taxa_lambda, size=(simulacoes, n_clt))
medias_amostrais = matriz_exponencial.mean(axis=1)

print(f"Geradas {simulacoes} médias amostrais. Gerando gráfico de densidade...")

# Plotando o resultado para evidenciar a convergência assintótica
plt.figure(figsize=(8, 5))
sns.histplot(medias_amostrais, stat="density", color="#E69F00", bins=60, alpha=0.7)
sns.kdeplot(medias_amostrais, color="#56B4E9", linewidth=2.5)
plt.title(f"Teorema Central do Limite (Tamanho da Amostra n = {n_clt})", fontsize=14)
plt.xlabel("Médias Amostrais (Distribuição Original: Exponencial)")
plt.ylabel("Densidade")
plt.show()

```

--- 1. Simulação da Distribuição de Poisson ---

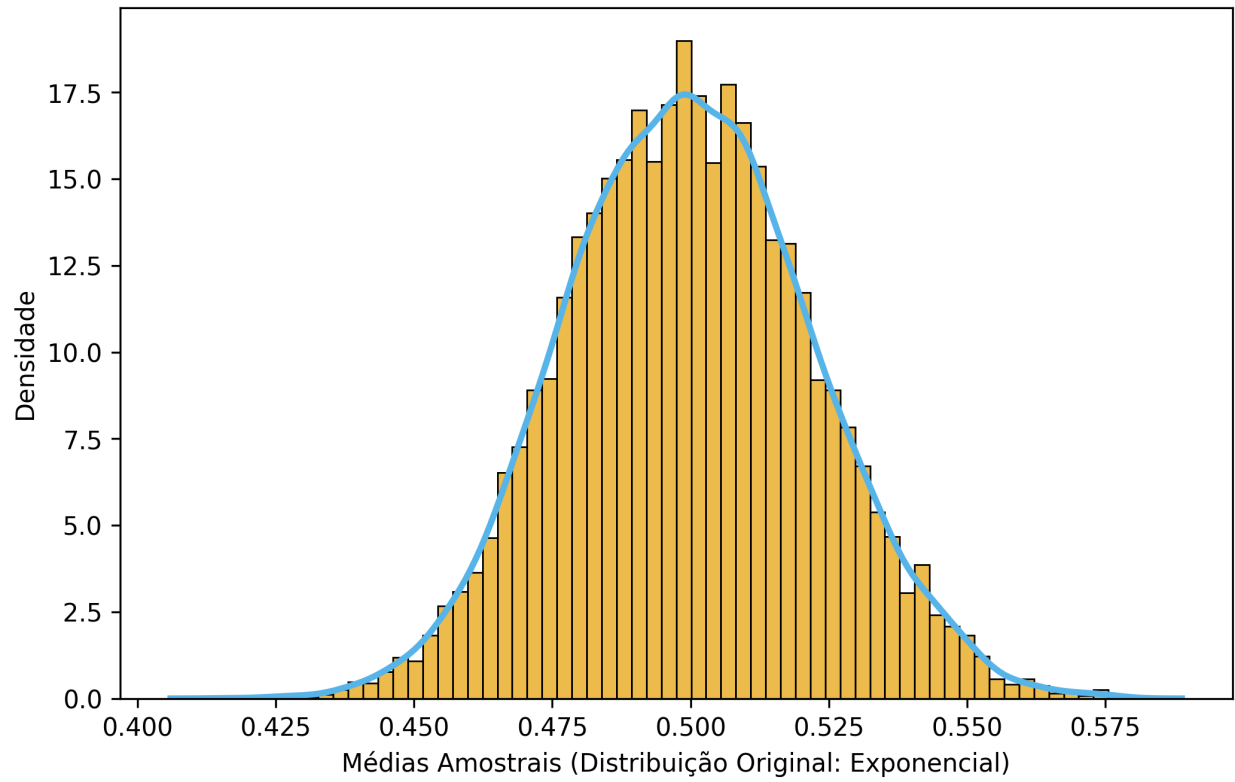
Soma Empírica do array: 350581

Valor Teórico Esperado (n * θ): 350000.0

--- 2. Teorema Central do Limite (Convergência para Normal) ---

Geradas 10000 médias amostrais. Gerando gráfico de densidade...

Teorema Central do Limite (Tamanho da Amostra $n = 500$)



6 Web Scraping

Até o momento, o fluxo analítico apresentado pressupôs que os dados já estivessem estruturados e acessíveis em formatos padronizados (como arquivos *CSV*). Contudo, no cenário contemporâneo da Ciência de Dados, a maior parte das informações geradas diariamente não se encontra tabularmente organizada, mas sim distribuída em milhões de páginas na internet. A técnica de acessar, processar e estruturar esses dados de forma automatizada é conhecida como Web Scraping (raspagem de dados).

6.1 Fundamentos da *Web* e Ética na Extração

Para extrair informações da internet, é essencial compreender a arquitetura da *web*. Quando acessamos um site, nosso computador (o cliente) envia uma requisição, através do protocolo *HTTP* (*Hypertext Transfer Protocol*) para um computador remoto (o servidor). Na raspagem de dados, a requisição mais comum é o método *GET*, utilizado para solicitar a leitura de uma página (Mitchell, 2018).

Se a requisição for bem-sucedida, o servidor retorna um documento de texto escrito em *HTML* (*HyperText Markup Language*). O entendimento da estrutura dos códigos *HTML* sobre os quais as páginas da *web* são construídas é exatamente o que nos permite realizar o processo de raspagem de dados.

O *HTML* não é uma linguagem de programação, mas uma linguagem de marcação, que organiza o conteúdo através de etiquetas (*tags*), como `<p>` para parágrafos, `<h1>` para títulos e `<table>` para tabelas. Esse documento é estruturado de forma hierárquica, formando uma árvore de elementos conhecida como *DOM*¹. É através dos atributos dessas tags (especialmente `class` e `id`) que os algoritmos de scraping conseguem localizar exatamente qual pedaço de texto ou número deve ser extraído.

Dessa forma, as requisições são guiadas justamente por essas *tags* que servem como um “mapa” da *internet*. Elas nos permitem generalizar os códigos e buscar, teoricamente, de forma ilimitada pela *web*, uma vez que não precisamos conhecer exatamente os textos e seus caracteres para saber em que local da página buscar, necessitamos apenas saber qual o local mais provável de se encontrar um conteúdo dentro da estrutura *HTML*.

¹O *DOM* (*Document Object Model*) é uma interface de programação que representa o documento *HTML* como uma árvore lógica. Ele permite que linguagens de programação, como o *Python* ou o *JavaScript*, naveguem, acessem e modifiquem o conteúdo, a estrutura e o estilo da página *web*.

6.1.1 Ética e Boas Práticas na Raspagem

A raspagem de dados apresenta desafios que vão muito além da capacidade computacional e da abrangência lógica do código. Um número muito alto de requisições pode sobrecarregar o servidor alvo, podendo até mesmo levá-lo a cair, o que configura um ataque de negação de serviço (DDoS) accidental. Para garantir uma conduta ética, o analista de dados deve respeitar três pilares fundamentais (Mitchell, 2018):

- Inspeção do arquivo `robots.txt`: Antes de raspar um domínio, deve-se verificar este arquivo público, que contém as diretrizes do administrador sobre quais páginas podem ou não ser acessadas por robôs (acessível via `(nome_do_site.com/robots.txt)`). Por exemplo, se quiser raspar dados da *Gov*, deve acessar gov.br/robots.txt.
- Identificação do *User-Agent*: Por padrão, a biblioteca de requisições do Python se identifica como um robô genérico. É recomendável alterar o *User-Agent* no cabeçalho da requisição para um identificador real (como o de um navegador comum) e, em projetos acadêmicos ou comerciais, fornecer um contato válido para o administrador do servidor.
- Implementação de *Delays*: O código deve incluir pausas intencionais (usando a função `time.sleep()`) entre as requisições, simulando a velocidade de leitura de um ser humano e preservando a largura de banda do servidor acessado.

6.2 O Ecossistema de Raspagem Estática

Uma vez entendidas as questões éticas da raspagem de dados e estruturais da *web*, podemos partir para a operacionalização da coleta de dados. Primeiramente, veremos o caso das páginas cujo conteúdo *HTML* é entregue, em sua totalidade, logo na requisição inicial. Nesse caso, dizemos que a extração está sendo feita sobre páginas estáticas.

O *Python* conta com duas bibliotecas clássicas para a raspagem estática: a `requests` e a `BeautifulSoup`. A biblioteca `requests` é responsável por gerenciar a comunicação *HTTP*, nos permitindo baixar os conteúdos de uma página através de comandos simples e diretos. Um aspecto crucial ao utilizar essa ferramenta é a validação do código de status da resposta (*status code*). Um código 200 indica que a requisição foi bem-sucedida, enquanto códigos na casa dos 400 (como o famoso 404 *Not Found* ou 403 *Forbidden*) sinalizam, respectivamente, que a página não existe ou que o servidor bloqueou o acesso, possivelmente identificando a automação (Mitchell, 2018).

Uma vez que o código *HTML* bruto é capturado com sucesso, ele é armazenado no computador apenas como uma longa e contínua cadeia de texto (*string*). Analisar dados diretamente desse formato seria extremamente complicado e suscetível a erros. É neste contexto que a biblioteca `BeautifulSoup` (importada do pacote `bs4`) se faz necessária.

Ela atua como um “parseador” (*parser*), que analisa o texto bruto e o converte na árvore hierárquica do *DOM*, discutida na seção anterior. Ao transformar a *string* estática em um objeto estruturado e navegável, o `BeautifulSoup` disponibiliza métodos otimizados para que possamos buscar dados de forma cirúrgica, utilizando os nomes das *tags* e seus respectivos atributos.

6.3 Navegação e Extração de Elementos (Conteúdo Estático)

Com a página da *web* “traduzida” através do BeautifulSoup, o conteúdo que antes era confuso agora é estruturado de maneira hierárquica e perfeitamente mapeada. Neste contexto, nosso desafio passa a ser a localização exata da informação desejada dentro dessa árvore de elementos (*tags*).

Para navegar por essa estrutura, a biblioteca fornece diversos métodos, sendo os mais proeminentes o `find()` e o `find_all()`. O método `find()` percorre o documento e retorna a primeira ocorrência de uma *tag* específica que satisfaça os critérios definidos. Já o `find_all()` varre toda a página e retorna uma coleção (uma lista estruturada) contendo todos os elementos que correspondam a busca.

Além da busca genérica pelo nome da *tag* (como procurar por todos os parágrafos `<p>`), podemos afunilar a filtragem através dos atributos da *tag*, especialmente as classes, utilizando o parâmetro `class_` (com um *underline* no final, para não conflitar com a palavra reservada do *Python*). Caso o analista possua familiaridade com desenvolvimento *web* (um dos grandes diferenciais que um analista pode ter no contexto da raspagem de dados), o método `select()` permite o uso de seletores *CSS* diretos, oferecendo uma sintaxe ainda mais concisa e aumentando as possibilidades de caminhos para as buscas.

Uma vez que o elemento correto é isolado, a extração da informação útil exige a separação entre o dado e o código *HTML*. O texto visível aos usuários pode ser extraído acessando a propriedade `.text` do objeto. Já quando o objetivo é coletar *links* ou caminhos de arquivos, a informação desejada não se encontra no texto livre, mas embutida dentro da própria *tag* (no atributo `href`), sendo necessário extraí-la utilizando o método `.get('href')`.

O *script* a seguir ilustra a aplicação prática desses conceitos utilizando um documento *HTML* simulado. Essa abordagem garante a reprodutibilidade metodológica do exemplo, eliminando a dependência de conexão com servidores externos.

```
from bs4 import BeautifulSoup

# Simulando o HTML estático de uma página estruturada
html_simulado = """
<html>
  <body>
    <h1 class="titulo-principal">Estatísticas Demográficas</h1>
    <p class="resumo">Este é um resumo dos dados do censo.</p>
    <div id="dados-estatisticos">
      <a href="https://exemplo.com/tabela_norte.csv"
        class="link-dados">Dados Região Norte</a>
      <a href="https://exemplo.com/tabela_sul.csv"
        class="link-dados">Dados Região Sul</a>
    </div>
  </body>
</html>
"""

# Instanciando o parser para criar a árvore DOM
```

```
sopa = BeautifulSoup(html_simulado, 'html.parser')

# Usando find() para extrair o texto de uma classe específica
titulo = sopa.find('h1', class_='titulo-principal').text
print(f"Título da Página: {titulo}\n")

# Usando find_all() e iterando para extrair e formatar múltiplos links
tags_link = sopa.find_all('a', class_='link-dados')

print("Arquivos Disponíveis para Download:")
for tag in tags_link:
    texto_link = tag.text          # Extrai o texto visível
    url = tag.get('href')          # Extrai o link oculto no atributo
    print(f"- {texto_link}: {url}")
```

Título da Página: Estatísticas Demográficas

Arquivos Disponíveis para Download:

- Dados Região Norte: https://exemplo.com/tabela_norte.csv
- Dados Região Sul: https://exemplo.com/tabela_sul.csv

6.4 Raspagem de Estruturas Tabulares (Integração com Pandas)

Embora os métodos `find()` e `find_all()` do `BeautifulSoup` sejam suficientes para extrair qualquer elemento isolado de uma página, extrair uma estrutura tabular completa (a *tag* `<table>`) elemento por elemento seria um processo redundante e propenso a erros. Para isso, o *Python* oferece uma ponte direta o *HTML* tabular e a estrutura de um `DataFrame`, através da biblioteca `pandas`, da qual já falamos anteriormente.

Neste contexto, destaca-se a função `pd.read_html()`, que automatiza o processo como um todo: ao receber um trecho de código *HTML*, ela localiza internamente todas as *tags* `<table>` presentes, interpreta suas linhas (`<td>`), e devolve uma lista de `DataFrames` prontos para uso - um para cada tabela encontrada no documento (McKinney, 2022).

É importante destacar, por razões de segurança, em versões mais recentes do `pandas` a função não aceita mais *strings* de *HTML* diretamente, exigindo que o texto seja envolvido pela classe `StringIO` do módulo `io`, nativo do *Python*, simulando um arquivo em memória. Esse mesmo mecanismo se aplica quando o *HTML* já foi previamente capturado por uma requisição `requests`, sendo a prática recomendada em qualquer cenário de raspagem real.

O script a seguir demonstra esse fluxo completo, partindo de uma tabela *HTML* simulada até a obtenção de um `DataFrame` estruturado, pronto para as etapas de tratamento e análise discutidas no Capítulo 4.

```

import pandas as pd
from io import StringIO

# Simulando uma tabela HTML, como a que poderia ser retornada por uma requisição
html_tabela = """
<table>
  <caption>Indicadores Regionais</caption>
  <tr>
    <th>Região</th>
    <th>População</th>
    <th>PIB (em bilhões)</th>
  </tr>
  <tr>
    <td>Norte</td>
    <td>18000000</td>
    <td>250</td>
  </tr>
  <tr>
    <td>Sul</td>
    <td>30000000</td>
    <td>800</td>
  </tr>
</table>
"""

# read_html() exige que a string seja envolvida por StringIO
tabelas_encontradas = pd.read_html(StringIO(html_tabela))

# A função retorna uma lista de DataFrames (uma por tabela na página)
df_indicadores = tabelas_encontradas[0]

print(f"Número de tabelas encontradas: {len(tabelas_encontradas)}\n")
print(df_indicadores)

```

Número de tabelas encontradas: 1

	Região	População	PIB (em bilhões)
0	Norte	18000000	250
1	Sul	30000000	800

Como resultado, obtemos diretamente um **DataFrame** com tipagem já inferida (valores numéricos reconhecidos como tais), eliminando por completo a necessidade de laços de repetição manuais para percorrer células — o “pulo do gato” analítico mencionado anteriormente. A partir deste ponto, o ‘df_indicadores’ já pode ser submetido a qualquer uma das ferramentas de *Data Wrangling* apresentadas no Capítulo 4, como `.describe()`, `dropna()` ou `merge()` com outras bases.

Vale ressaltar que essa abordagem tem uma limitação natural: ela funciona apenas quando a informação já está de fato estruturada em uma *tag* `<table>` no *HTML*. Muitas páginas modernas

simulam visualmente uma tabela usando `<div>`s estilizados via *CSS*, caso em que a extração exige o uso combinado de `find_all()` — técnica que retomaremos no estudo de caso da seção 6.6.

6.5 Raspagem Dinâmica e Automação com Selenium

As técnicas apresentadas nas seções anteriores partem de uma premissa importante: o conteúdo relevante já está presente no HTML devolvido pela requisição inicial. Essa premissa, no entanto, não se sustenta para uma parcela crescente da web moderna. Em muitas aplicações, o `requests` retorna apenas um esqueleto HTML praticamente vazio, enquanto o conteúdo real (tabelas, listas, resultados de busca) é inserido dinamicamente na página por código *JavaScript*, executado somente depois que um navegador carrega e interpreta a página por completo.

Como o `BeautifulSoup` não executa *JavaScript* — ele apenas interpreta o texto estático que recebe —, essas páginas exigem uma abordagem estrutural diferente: em vez de simular uma requisição, é necessário automatizar um navegador real. É exatamente essa a função da biblioteca `Selenium`, originalmente concebida para testes automatizados de software, mas amplamente adotada pela comunidade de dados como solução para raspagem dinâmica (Mitchell, 2018).

6.5.1 Configuração do Webdriver

O `Selenium` opera controlando um *webdriver*: um executável que atua como intermediário entre o código Python e um navegador instalado na máquina (como o Google Chrome). Em versões mais antigas da biblioteca, era necessário baixar manualmente o executável correspondente à versão exata do navegador instalado — um processo frágil e propenso a quebras a cada atualização. Desde a versão 4.6, o `Selenium` incorpora o *Selenium Manager* (Selenium Project, 2026), um mecanismo interno que detecta automaticamente a versão do navegador instalado e baixa o *driver* compatível, dispensando essa configuração manual.

```
from selenium import webdriver
from selenium.webdriver.chrome.options import Options

# Configurando opções do navegador
opcoes = Options()
opcoes.add_argument("--headless") # Executa sem abrir uma janela visível

# O Selenium Manager cuida automaticamente do driver compatível
navegador = webdriver.Chrome(options=opcoes)

navegador.get("https://www.selenium.dev/selenium/web/web-form.html")
print(navegador.title)

navegador.quit() # Encerra o processo do navegador
```

O parâmetro `--headless` merece destaque: ele instrui o navegador a operar em segundo plano, sem renderizar uma interface gráfica visível. Essa é a configuração padrão para scripts de raspagem em produção, já que reduz o consumo de recursos computacionais.

6.5.2 Interação com a Página

Diferentemente do BeautifulSoup, que apenas lê uma “fotografia” estática do *HTML*, o Selenium permite simular ações humanas dentro do navegador controlado. Os métodos mais utilizados para localizar elementos são agrupados na classe `By`, que oferece estratégias de busca (por ID, por `CSS_SELECTOR`, por `XPATH`, entre outras). Uma vez localizado, o elemento pode ser manipulado através de métodos como `.click()`, para simular um clique, e `.send_keys()`, para preencher campos de formulário.

```
from selenium.webdriver.common.by import By

# Localizando um campo de texto pelo nome e preenchendo-o
campo_texto = navegador.find_element(By.NAME, "my-text")
campo_texto.send_keys("Estatística com Python")

# Localizando e clicando no botão de envio
botao_envio = navegador.find_element(By.CSS_SELECTOR, "button[type='submit']")
botao_envio.click()
```

6.5.3 Espera Explícita e Implícita

Talvez o desafio mais característico da raspagem dinâmica seja o problema da sincronização: o código *Python* é executado em milissegundos, mas o carregamento de conteúdo via JavaScript pode levar segundos, dependendo da conexão e do servidor. Tentar extrair um elemento antes que ele tenha sido efetivamente renderizado resulta no erro `NoSuchElementException`.

Para lidar com essa assincronia, o Selenium oferece dois mecanismos de espera. A espera implícita, configurada uma única vez com `navegador.implicitly_wait(segundos)`, instrui o driver a aguardar automaticamente, por um tempo máximo definido, antes de lançar um erro em qualquer tentativa de busca de elemento. Já a espera explícita, através da classe `WebDriverWait`, é uma abordagem mais precisa e recomendada em cenários de produção: ela permite pausar a execução até que uma condição específica seja satisfeita (como a presença ou a clicabilidade de um elemento), evitando tanto erros prematuros quanto esperas desnecessariamente longas.

```
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

# Aguarda até 10 segundos pela presença do elemento antes de prosseguir
espera = WebDriverWait(navegador, 10)
resultado = espera.until(
    EC.presence_of_element_located((By.ID, "resultado-busca"))
)
```

Diferentemente dos exemplos das seções 6.3 e 6.4, os blocos de código desta seção foram marcados com `#| eval: false` no cabeçalho do *chunk*. Isso ocorre porque a execução do Selenium depende de um navegador instalado e de conexão ativa com a internet no momento da renderização do documento — uma dependência externa que comprometeria a reprodutibilidade automática do

livro em ambientes de compilação (CI/CD, por exemplo). O código permanece correto e executável na máquina do leitor, mas não será rodado automaticamente ao gerar o PDF/HTML do livro.

6.6 Estudos de Caso Completos

Encerramos este capítulo consolidando, em dois estudos de caso de ponta a ponta, tudo o que foi discutido ao longo das seções anteriores: os fundamentos éticos da seção 6.1, a raspagem estática das seções 6.2 a 6.4 e a automação dinâmica da seção 6.5. O primeiro caso ilustra o fluxo estático completo sobre uma página real; o segundo, o fluxo dinâmico com Selenium.

6.6.1 Caso 1 (Estático): Extraindo uma Tabela Real com Requests e BeautifulSoup

Diferentemente do exemplo controlado da seção 6.3, este estudo de caso aplica o fluxo completo sobre uma página real da internet, respeitando as boas práticas discutidas na seção 6.1: identificação via **User-Agent** e verificação do código de status antes de prosseguir com o processamento.

Note que, ao contrário da função `pd.read_html()` apresentada na seção 6.4 (que automatiza toda a extração de uma tabela HTML), este exemplo demonstra o processo manual de construção de um `DataFrame` a partir dos elementos `find_all()`. Essa abordagem é necessária quando a estrutura da tabela é irregular demais para a inferência automática do pandas, ou quando se deseja um controle mais granular sobre a limpeza de cada célula durante a própria extração.

```
import requests
from bs4 import BeautifulSoup
import pandas as pd

url = "https://en.wikipedia.org/wiki/List_of_countries_by_GDP_(nominal)"
cabecalhos = {"User-Agent": "Projeto-IC-UFJF-Gabriel (contato: gabrielaffonso@gmail.com)"}

resposta = requests.get(url, headers=cabecalhos)

if resposta.status_code == 200:
    sopa = BeautifulSoup(resposta.text, "html.parser")
    tabela = sopa.find("table", class_="wikitable")

    linhas_html = tabela.find_all("tr")
    cabecalho_tabela = [th.text.strip() for th in linhas_html[0].find_all("th")]

    linhas_dados = []
    for linha in linhas_html[1:]:
        celulas = linha.find_all(["td", "th"])
        valores = [celula.text.strip() for celula in celulas]
        if valores:
            linhas_dados.append(valores)

    df_pib = pd.DataFrame(linhas_dados, columns=cabecalho_tabela[:len(linhas_dados[0])])
```

```

df_pib.to_csv("pib_paises.csv", index=False, encoding="utf-8")

print(f"Tabela extraída com sucesso: {df_pib.shape[0]} linhas.")
print(df_pib.head())
else:
    print(f"Falha na requisição. Status code: {resposta.status_code}")

```

Tabela extraída com sucesso: 222 linhas.

	Country/Territory	IMF(2026)[1]	World Bank(2025)[6]	United Nations(2024)[7]
0	World	126,295,331	118,350,166	111,565,144
1	United States	32,383,920	30,769,700	29,298,000
2	China[n 1]	20,851,593	19,498,039	18,743,802
3	Germany	5,452,858	5,050,923	4,659,929
4	Japan	4,379,253	4,435,163	4,026,211

É importante que o leitor esteja ciente de uma realidade prática da raspagem de dados: páginas reais frequentemente possuem estruturas irregulares (células mescladas, cabeçalhos aninhados, notas de rodapé embutidas), exigindo ajustes manuais que um exemplo controlado não demonstra. Esse é, inclusive, um dos motivos pelos quais scripts de raspagem quebram com o tempo — a estrutura HTML de uma página está sujeita a mudanças a qualquer momento, sem aviso prévio (Mitchell, 2018).

6.6.2 Caso 2 (Dinâmico): Automatizando a Coleta com Selenium

Para o estudo de caso dinâmico, utilizamos um ambiente construído especificamente para fins didáticos de raspagem de dados — o site *Quotes to Scrape* (em sua versão de rolagem infinita), que carrega novas citações via *JavaScript* à medida que o usuário rola a página para baixo. Por ser um ambiente público criado justamente para práticas de scraping, seu uso é livre de questões éticas ou de sobrecarga indevida de servidores de terceiros.

O *script* abaixo simula a rolagem da página, aguarda o carregamento do conteúdo dinâmico e extrai o texto de cada citação junto ao seu respectivo autor.

```

from selenium import webdriver
from selenium.webdriver.chrome.options import Options
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
import time
import pandas as pd

opcoes = Options()
opcoes.add_argument("--headless")

navegador = webdriver.Chrome(options=opcoes)
navegador.get("https://quotes.toscrape.com/scroll")

```

```

espera = WebDriverWait(navegador, 10)

# Simulando rolagem da página para acionar o carregamento dinâmico
for _ in range(5):
    navegador.execute_script("window.scrollTo(0, document.body.scrollHeight);")
    time.sleep(1.5) # Aguarda o carregamento via JavaScript

# Espera explícita pela presença de ao menos uma citação
espera.until(EC.presence_of_element_located((By.CLASS_NAME, "quote")))

citacoes_elementos = navegador.find_elements(By.CLASS_NAME, "quote")

dados_citacoes = []
for elemento in citacoes_elementos:
    texto = elemento.find_element(By.CLASS_NAME, "text").text
    autor = elemento.find_element(By.CLASS_NAME, "author").text
    dados_citacoes.append({"citacao": texto, "autor": autor})

navegador.quit()

df_citacoes = pd.DataFrame(dados_citacoes)
df_citacoes.to_csv("citacoes_dinamicas.csv", index=False, encoding="utf-8")

print(f"Total de citações coletadas: {len(df_citacoes)}")
print(df_citacoes.head())

```

Total de citações coletadas: 100

		citacao	autor
0	"The world as we have created it is a process ...	Albert Einstein	
1	"It is our choices, Harry, that show what we t...	J.K. Rowling	
2	"There are only two ways to live your life. On...	Albert Einstein	
3	"The person, be it gentleman or lady, who has ...	Jane Austen	
4	"Imperfection is beauty, madness is genius and...	Marilyn Monroe	

Note que a combinação de `execute_script()` (para simular a rolagem via JavaScript) com a espera explícita `WebDriverWait` resume, na prática, os dois grandes desafios da raspagem dinâmica discutidos na seção 5.5: a necessidade de interagir com a página e a necessidade de sincronizar essa interação com o tempo de resposta do servidor.

7 Livros Digitais: Estruturação de relatórios dinâmicos via *Quarto*

No ecossistema da Ciência de Dados, o fluxo de trabalho não se encerra na obtenção do modelo ou na geração da tabela final. A reprodutibilidade da pesquisa e a clareza na transmissão do conhecimento exigem que os resultados sejam integrados a relatórios dinâmicos, nos quais o código que gera a análise e o texto que a interpreta coexistam no mesmo ambiente. É neste contexto que ferramentas de autoria científica se tornam indispensáveis.

7.1 A Evolução para o *Quarto*

Historicamente, a comunidade estatística estabeleceu um padrão na geração de relatórios dinâmicos através do *R Markdown*. No entanto, com a consolidação do *Python* e de outras linguagens como o *Julia* no cenário analítico, o acoplamento exclusivo ao ecossistema do *R* tornou-se uma limitação.

O *Quarto* surge como a evolução natural e o sucessor de nova geração do *R Markdown*. Desenvolvido pela *Posit*, trata-se de um sistema de publicação científica e técnica de código aberto construído sobre o *Pandoc*. Sua principal revolução é ser intrinsecamente agnóstico em relação à linguagem de programação: ele é capaz de compilar scripts nativos de *Python* (através do motor *Jupyter*), *R* (via *Knitr*) e *Julia*, transformando arquivos de texto puro (*.qmd*) em documentos de altíssima qualidade visual, como relatórios em PDF, dashboards em HTML, e livros digitais completos.

Adotar o *Quarto* significa abandonar a prática de copiar e colar gráficos soltos em editores de texto tradicionais — um processo manual, sujeito a erros e impossível de ser atualizado automaticamente quando a base de dados muda.

7.2 A Estrutura de Metadados (O Cabeçalho *YAML*)

Todo documento *Quarto* é dividido fundamentalmente em três partes: o cabeçalho de metadados, o texto em marcação (*Markdown*) e os blocos de código executáveis (*chunks*). O controle do comportamento do documento ocorre primordialmente no topo do arquivo, em uma seção isolada por três traços (*---*), escrita em uma linguagem de serialização chamada *YAML* (*YAML Ain't Markup Language*).

É no *YAML* que definimos as diretrizes globais do projeto: o motor de renderização, o formato de saída, as opções padrão de execução dos códigos e a padronização do leiaute.

Quando trabalhamos com projetos escaláveis ou livros com múltiplos capítulos, é imperativo garantir uma identidade visual coesa e um rastreamento interno eficiente. Para isso, configuramos referências customizadas. O exemplo abaixo demonstra um cabeçalho *YAML* profissional, pronto para produção, que aplica a configuração de estilo definida para os seus projetos (que chamaremos de “Meu template”) e estabelece o identificador interno exigido (que será “m_template”):

Vamos entender as chaves deste cabeçalho:

- A subchave `template` sob `format`: garante que, independentemente do capítulo renderizado, a identidade visual carregue estritamente as customizações do “Meu Template”.
- A chave `project`: `id` atua como um rastreador interno nos bastidores da compilação. Isso é especialmente útil em projetos grandes, onde extensões e scripts secundários precisam referenciar a estrutura matriz do projeto.
- A chave `execute` define um comportamento global para os chunks de *Python*. Ao definir `warning: false` e `message: false`, garantimos que o relatório final seja limpo e legível, livre de alertas de compilação ou mensagens de carregamento de pacotes que não interessam ao leitor final.

7.3 *Templates* e Padronização Visual

Quando um projeto de Ciência de Dados ganha escala, ultrapassando a barreira de um único script e se tornando uma obra de múltiplos capítulos ou relatórios periódicos, a manutenção de uma identidade visual consistente torna-se primordial. O *Quarto* oferece suporte nativo a temas *Bootswatch* (como *cosmo*, *flatly* ou *journal*), mas a verdadeira robustez reside no uso de *Templates* customizados e extensões.

Através da interface de linha de comando (CLI) do *Quarto*, é possível instalar extensões de formatação que encapsulam folhas de estilo (*CSS* ou *SCSS*), arquivos de marcação parcial e lógicas de numeração. Ao definirmos um modelo próprio no *YAML* (como o Meu Template configurado na seção anterior), garantimos que todos os títulos, espaçamentos, fontes e blocos de código gerados no projeto obedeçam estritamente ao mesmo padrão visual.

Essa modularidade é o que permite que uma equipe de dados se preocupe exclusivamente com a lógica estatística e o conteúdo, enquanto o template assume, nos bastidores, a responsabilidade de manter a apresentação com qualidade de publicação. O domínio dessas customizações avançadas e da estruturação de projetos e livros digitais no ecossistema do *Quarto* deve muito às contribuições da comunidade de desenvolvedores e educadores, destacando-se materiais práticos e tutoriais de referência como os desenvolvidos por Hall (2026).

7.4 Elementos Científicos e Tabelas Complexas

A produção de documentos técnicos institucionais — como, por exemplo, a publicação de um Sumário Brasileiro de Touros da EMBRAPA ou de boletins epidemiológicos governamentais — impõe desafios de formatação que vão muito além da exportação padrão de um *DataFrame*. Nesses cenários, a comunicação exige diagramação avançada, mapas interativos e matemática rigorosa.

O Quarto se destaca ao permitir que o texto e as saídas das linguagens de programação interajam perfeitamente com a tipografia científica:

- Tabelas Estruturadas e Índices Complexos

A função padrão de exibição de dados (`print()`) é insuficiente para publicações oficiais. Um caso clássico é a apresentação de tabelas com múltiplos níveis hierárquicos, como as necessárias para detalhar índices específicos de performance (por exemplo, a estruturação de uma tabela para o índice IPGL, do Sumário Brasileiro de Touros da EMBRAPA). O *Quarto* integra-se perfeitamente a pacotes de estilização avançada. Se o código base for *Python*, bibliotecas como `great_tables` ou a própria estilização nativa do *pandas* (`df.style`) são renderizadas em *HTML* ou *LaTeX* estrito. Se o ambiente envolver integração com *R*, o pacote `kableExtra` permite a construção de cabeçalhos agrupados e notas de rodapé essenciais para relatórios institucionais.

- Renderização de Mapas e Gráficos Interativos

A integração espacial é frequentemente necessária em projetos de larga escala. Se o relatório exigir o mapeamento de informações geoespaciais, o Quarto compila mapas gerados pelo pacote `leaflet` diretamente no livro digital. No formato *HTML*, os mapas mantêm sua interatividade (zoom e pan); no formato PDF, o motor captura um “print” de alta qualidade.

- Notação Matemática e Metodologia Estatística

Um relatório científico frequentemente precisa explicitar suas escolhas metodológicas sob a ótica matemática. O *Quarto* suporta a sintaxe *LaTeX* nativamente. Isso significa que podemos “escrever matemática” de forma livre, como se estivéssemos no papel, como por exemplo a especificação formal de matrizes de contrastes ortogonais, os parâmetros de ponderação (como os Ptg’s de um projeto) e as equações de inferência, garantindo que o leitor entenda não apenas o resultado do código, mas a teoria por trás dele.

Um bloco de equação no *Quarto* é escrito delimitando a matemática com duplos cifrões `$$`:

A equação do modelo linear múltiplo, por exemplo, é definida por:

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \cdots + \beta_k X_k + \epsilon$$

7.5 Compilação de Livros Digitais (O arquivo `_quarto.yml`)

Até aqui, tratamos os capítulos como arquivos `.qmd` independentes. No entanto, a verdadeira força do Quarto para publicações extensas reside na sua capacidade de orquestrar múltiplos arquivos em uma única obra coesa (um *book*).

Essa orquestração não é feita dentro dos capítulos, mas sim em um arquivo de configuração mestre chamado `_quarto.yml`, localizado na raiz do diretório do projeto. É ele que dita a estrutura de navegação, a ordem de compilação e as configurações globais.

Quando executamos o comando de renderização, o compilador lê primeiramente este arquivo. Se ele indicar que o projeto é do tipo *book*, o *Quarto* unificará os capítulos, criará um sumário global interativo (para *HTML*) ou um índice analítico (para PDF), e resolverá automaticamente todas as referências cruzadas ao longo do texto.

Abaixo, temos um exemplo prático da estrutura de um arquivo `_quarto.yml` para a compilação de um livro:

Com essa arquitetura, a adição de um novo capítulo ou a reorganização da ordem de leitura exige apenas a modificação de uma linha de texto na lista `chapters`. Essa abordagem elimina o trabalho braçal de diagramação e permite que o foco do analista permaneça onde realmente importa: na estatística, na análise e na comunicação clara dos resultados.

8 Referências

HALL, Meghan. **Meghan Hall - Data Analysis, R, and Quarto.**, 2026. Disponível em: <<https://meghan.rbind.io>>

HARRIS, Charles R. *et al.* Array programming with NumPy. **Nature**, v. 585, n. 7825, p. 357–362, 2020.

HUNTER, John D. Matplotlib: A 2D graphics environment. **Computing in science & engineering**, v. 9, n. 3, p. 90–95, 2007.

LUTZ, Mark. **Learning Python: Powerful Object-Oriented Programming.** 5. ed. [S.l.]: O'Reilly Media, Inc., 2013.

MCKINNEY, Wes. **Python for Data Analysis: Data Wrangling with pandas, NumPy, and Jupyter.** 3. ed. [S.l.]: O'Reilly Media, Inc., 2022.

MITCHELL, Ryan. **Web Scraping with Python: Collecting More Data from the Modern Web.** 2. ed. [S.l.]: O'Reilly Media, Inc., 2018.

PETERS, Tim. **PEP 20: The Zen of Python.** [S.l.]: Python Software Foundation, 2004. Disponível em: <<https://peps.python.org/pep-0020/>>.

PYTHON SOFTWARE FOUNDATION. **The Python Tutorial: Data Structures.** [S.l.: S.n.].

PYTHON SOFTWARE FOUNDATION. **Built-in Types - Ranges.** [S.l.: S.n.].

PYTHON SOFTWARE FOUNDATION. **Built-in Functions.** [S.l.: S.n.].

SEBESTA, Robert W. **Conceitos de Linguagens de Programação.** 11. ed. [S.l.]: Bookman Editora, 2018.

SELENIUM PROJECT. **Selenium WebDriver Documentation.** [S.l.: S.n.].

VAN ROSSUM, Guido; WARSAW, Barry; COGHLAN, Nick. **PEP 8: Style Guide for Python Code.** [S.l.]: Python Software Foundation, 2001. Disponível em: <<https://peps.python.org/pep-0008/>>.

VANDERPLAS, Jake. **Python Data Science Handbook: Essential Tools for Working with Data.** [S.l.]: O'Reilly Media, Inc., 2016.

WASKOM, Michael L. seaborn: statistical data visualization. **Journal of Open Source Software**,

v. 6, n. 60, p. 3021, 2021.